

AI Agent as Data Scientist

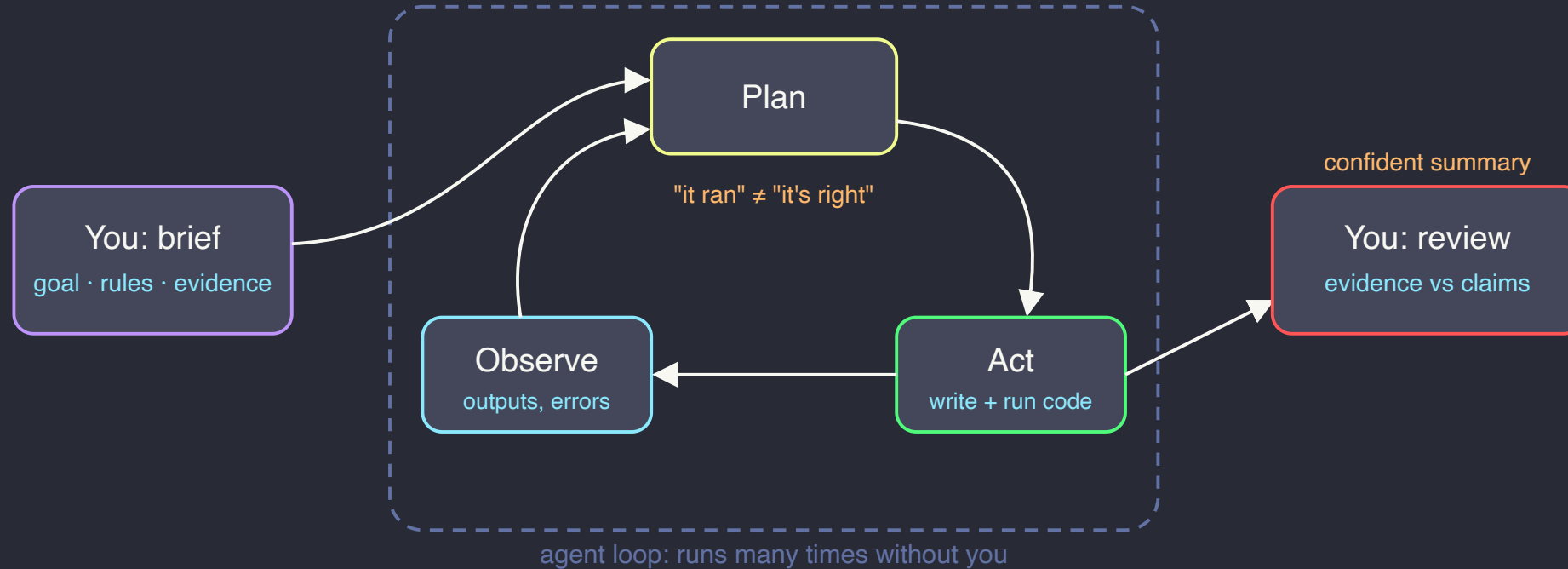


What Changed in the Last Two Years

- Coding agents now run **end-to-end** data science tasks from one instruction
 - Claude Code, GitHub Copilot agent mode (VS Code)
 - Data Science Agent in Google Colab, Copilot in Microsoft Fabric notebooks
- They read files, write code, **run it**, read errors and retry on their own
- Length of tasks agents can finish has been doubling roughly every 7 months
- Question is no longer "can it write the code?" but **"is the result right?"**

“ *Source: METR (2025), Measuring AI Ability to Complete Long Tasks — see References* ”

How an Agent Works



Where Agents Shine

- **Speed:** EDA, plots and a baseline model in minutes, not hours
- **Breadth:** tries five models and three encodings without getting bored
- **Boilerplate:** pipelines, data loading, docstrings, README
- **Unfamiliar libraries:** a working first draft in a new API
- **Explaining code:** walk through someone else's notebook line by line

Where Agents Fail Quietly

Failure	What it looks like
Drop before check	Removes the ID column first, so duplicate checks can only compare whole rows
Leaky split	Same customer or card on both sides of train/test
Wrong metric	Reports accuracy on imbalanced data, no baseline
Silent assumptions	Fills, filters or drops rows without saying so
Overclaiming	"Highly accurate, ready for production"
Specification gaming	Tunes on the test set, or weakens a check until it passes

Agents Optimise What You Ask For

- Brief says *"tell me how accurate it is"* → agent optimises and reports **accuracy**
- Business goal is *"flag $\geq 80\%$ of fraud **by value**"*, which the agent never heard
- Goodhart's law: **"when a measure becomes a target, it ceases to be a good measure"**
- Vague brief + confident agent = **a precise answer to the wrong question**

Scenario: Fraud Detection for a Retail Bank

- Fraud team reviews card transactions by hand, one row at a time
- Business goal: **flag at least 80% of confirmed fraud by dollar value**
 - with few enough false alerts for analysts to review
- Data: a public credit-card fraud sample, 8,262 transactions, label `is_fraud`
- Task for the agent: build the model and report how good it is

Live Demo: The One-Line Brief

Build a fraud detection model on `fraud_dataset.csv` and tell me how accurate it is.

- Dataset: the fraud sample from the scenario (link in the brief)
- Agent: whichever you have (Colab, Claude Code, Copilot agent mode, Fabric)
- Watch for: what it checks, what it drops, how it splits, what it claims

Live Demo (10 min)

- Run one-line brief in the Colab
 - screen shared
- Compare with another
 - agent_v1_output.ipynb (Claude Code)
- Dependent on agent, possible outcomes
 - may drop `trans_num`
 - may catch duplicates
 - may use stratified splits
 - may use accuracy metrics to report
 - ...

What the Agent Reported

“ *The model achieves **98.9% accuracy** with **99.5% recall** on fraud and **98.3% precision**. The model is highly accurate and **ready for production deployment**.* ”

- Code ran top to bottom with no errors
- Numbers are real and reproducible
- Dataset "perfectly balanced": 4,131 fraud vs 4,131 genuine

Hands-On: Review It Like a Pull Request (10 min)

- open `agent_review/agent_v1_output.ipynb` and `review_checklist.md`
- For each checklist section mark **OK / Problem / Not checked**, with evidence
- Write code to test your suspicions: you may add cells
- End with a verdict: **Approve / Request changes / Reject**
- Each group reports its single most important finding

Finding 1: Balance Manufactured by Copying

- 8,262 rows but only **5,415** distinct `trans_num`
- **2,847** exact duplicate rows, **every one a fraud**
- **86%** of test frauds have an identical twin in training
- Deduplicated: 4,131 genuine vs 1,284 fraud, so the baseline is **76%**, not 50%
- Agent dropped `trans_num` before looking, so the evidence disappeared

Finding 2: Same Card on Both Sides

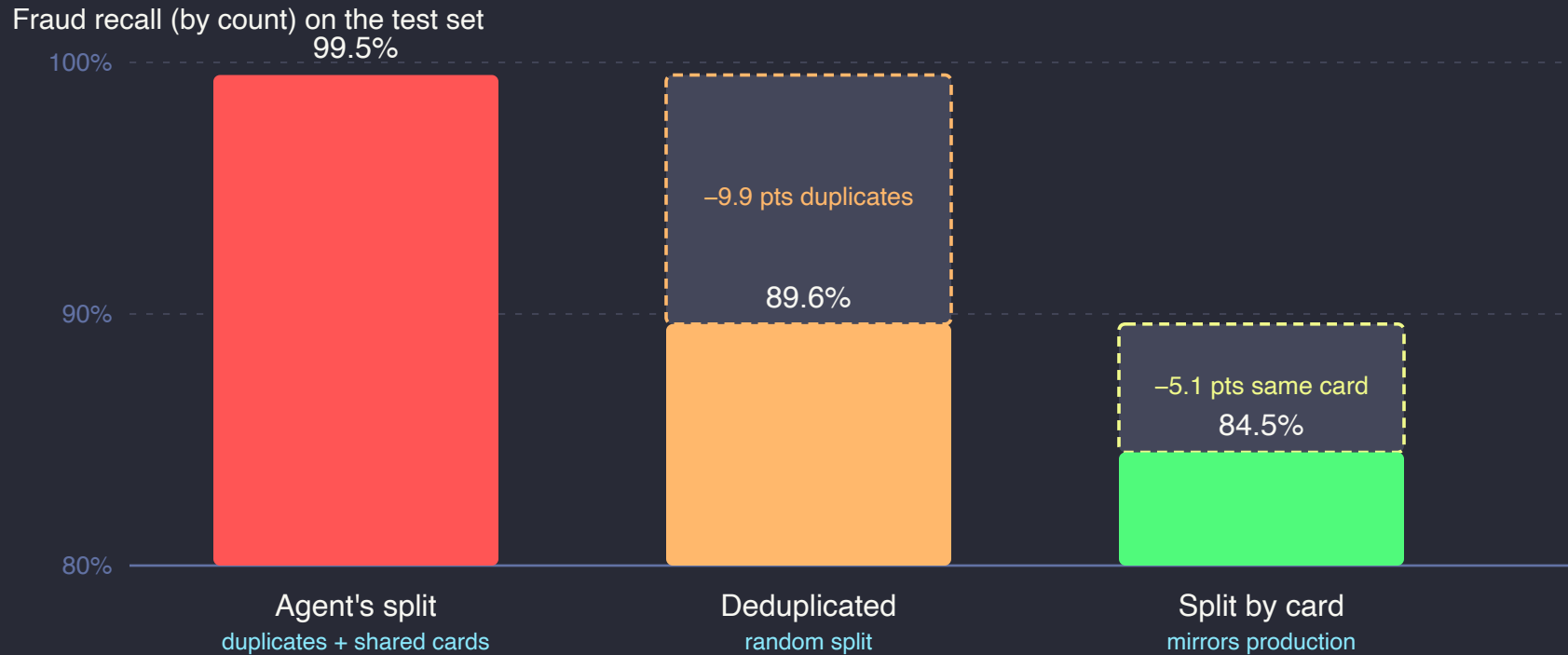
- Fraud comes in **bursts**: median of 10 fraud transactions per compromised card
- Random split: **99%** of compromised cards in test also appear as fraud in training
- Model learns *that card*, not *fraud*
- In production every stolen card is **new**,
 - so the test set must hold cards the model hasn't seen:
 - split by `cc_num` (`GroupShuffleSplit`)

```
from sklearn.model_selection import GroupShuffleSplit

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

train_idx, test_idx = next(gss.split(X, y, groups=df["cc_num"]))
```

How Much Was the Recall Inflated?



Finding 3: Report the Business Metric

Agent		After review
Headline	98.9% accuracy, "ready for production"	98.9% of fraud by value caught (goal ≥ 80%)
Fraud caught (count)	99.5%	84.5% on unseen cards; small frauds missed
Baseline	not reported	78% "always genuine"
Precision	98.3%	97.5%: few false alerts for analysts
Caveats	none	Oversampled source file, synthetic data, needs a time-based test

“ *Honest result is still good news, and now it is **defensible*** ”

Brief v2: Write It Like a Spec

- **Context:** who it's for, how fraud is handled today
- **Goal:** $\geq 80\%$ of fraud by value; report precision
- **Before modelling:** profile; explain before dropping
- **Evaluation rules:** split mirrors production; baseline beside every metric
- **Deliverables:** runnable notebook + summary of checks and risks
- **Forbidden:** claiming production-ready
- **Escalation:** stop and ask if data contradicts the brief

“ *Brief states the **goal and standard of evidence**; it never names the traps* ”

Demo: Re-run With Brief v2

- Same agent, same data, new brief
- Compare against v1:
 - Did it find the duplicates without being told?
 - Which split did it choose, and why?
 - Does the summary match the evidence?
- Review it with the same checklist: **better brief ≠ skip the review**

Supervising Agents: Working Patterns

- **Brief like a spec:** goal, constraints, evidence, done-criteria
- **Checkpoints:** "profile the data and stop", review, then "now model"
- **Ask for evidence, not adjectives:** "show the duplicate count", not "is the data clean?"
- **Version control everything:** agent edits are diffs you can read and revert
- **Independent check:** re-run key numbers yourself, or with a second agent as reviewer
- **Least privilege:** read-only data access; no deploy rights

What This Means for Your Career

Matters less

- Remembering syntax
- Writing boilerplate pipelines
- Speed at typing code

Matters more

- **Problem framing:** tie work to business value
- **Evaluation:** baselines, leakage, right metric
- **Domain knowledge:** "real fraud isn't 50%"
- **Review and communication:** defend a number to stakeholders

“ Agents raise the floor on producing analysis, and the bar on judging it ”

Key Takeaways

- Agents are fast and their code runs; **"it ran" is not "it's right"**
- They optimise **what you ask for**: put the business metric in the brief
- Review agent work like a pull request: **reproduce, data, split, metric, claims**
- Classic traps (duplicates, grouped leakage, accuracy on imbalance) still catch agents
- Your edge is **judgement**: framing, evaluation, domain knowledge

References — Official Documentation

Topic	Source
Claude Code	Anthropic. (2025). Claude Code overview . Claude Code Docs.
Copilot agent mode	Microsoft. (2025). Use agent mode in VS Code . Visual Studio Code Docs.
Copilot in Fabric	Microsoft. (2025). Overview of Copilot for Data Science and Data Engineering . Microsoft Learn.
Colab Data Science Agent	Google. (2025). Data Science Agent in Colab: The future of data analysis with Gemini . Google for Developers Blog.
Grouped splits	scikit-learn developers. (2025). Cross-validation iterators for grouped data . scikit-learn User Guide.

References – Further Reading

Topic	Source
METR (2025)	Kwa, T., West, B., Becker, J., et al. (2025). <i>Measuring AI Ability to Complete Long Tasks</i> . arXiv:2503.14499.
Kapoor & Narayanan (2023)	Kapoor, S., & Narayanan, A. (2023). <i>Leakage and the Reproducibility Crisis in Machine-Learning-Based Science</i> . <i>Patterns</i> , 4(9).
Kaufman et al. (2012)	Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). <i>Leakage in Data Mining: Formulation, Detection, and Avoidance</i> . <i>ACM TKDD</i> , 6(4).
Dataset origin	Shenoy, K. (2020). <i>Credit Card Transactions Fraud Detection Dataset</i> (simulated with Sparkov). Kaggle.

Q&A

- Materials: github.com/SophiArch/Briefings, folder `Agent/`
- Try at home: give **brief v2** to a different agent, then review it with the checklist